

Chapitre 3 — Réalisation et implémentation

III.1 — Introduction

Ce chapitre présente la réalisation concrète du système EmissionsIQ, en montrant comment les choix de conception définis au Chapitre 2 ont été traduits en code source opérationnel. L'objectif est double : d'une part, documenter les décisions d'implémentation qui dépassent la simple transposition du design — les compromis techniques réels, les paramètres extraits du code, les difficultés rencontrées — et, d'autre part, offrir au lecteur une vision complète et vérifiable du système réalisé. Le chapitre suit l'ordre de la chaîne de traitement physique : la station IoT embarquée sur ESP32, le pipeline de traitement backend avec ses trois points techniques non triviaux, le module d'intelligence artificielle, l'interface web du dashboard, et enfin le protocole de validation de bout en bout. Cette progression reflète la direction du flux de données : de la mesure physique par le capteur jusqu'à l'indicateur de performance affiché sur le tableau de bord du superviseur ANPE.

III.2 — Réalisation de la station IoT

Schéma de câblage et interfaces physiques

La station de surveillance physique est construite autour d'un microcontrôleur **ESP32 DevKit V1**, dont la double architecture cœur Xtensa LX6 et le module WiFi intégré en font le candidat idéal pour les applications IoT industrielles légères. Six capteurs sont raccordés à l'ESP32 selon trois bus distincts, dont les assignations de broches sont définies dans le fichier `pins.h`.

Le **MH-Z19B** (CO₂, technologie NDIR — Infrarouge Non Dispersif) communique via le bus UART Serial2, avec les broches GPIO16 (RX) et GPIO17 (TX). Ce capteur fonctionne sous 5 V (courant de pointe 150 mA) alors que les GPIO de l'ESP32 sont en logique 3,3 V : un convertisseur de niveau bidirectionnel est donc obligatoire entre les deux circuits pour éviter d'endommager l'ESP32.

Le **SDS011** (PM2.5/PM10, diffusion laser) communique via Serial1 sur les broches GPIO18 (RX) et GPIO19 (TX), également avec un convertisseur de niveau 5V→3,3V. Le mode de fonctionnement configuré est le mode « query » (interrogation à la demande), qui prolonge la durée de vie du ventilateur interne en évitant la rotation continue.

Le **SGP30** (COV / TVOC) communique via le bus I²C avec SDA sur GPIO21 et SCL sur GPIO22, à une tension d'alimentation de 3,3 V. Son adresse I²C est `0x58`. Deux résistances de tirage de 4,7 kΩ sont câblées entre chaque ligne SDA/SCL et le rail 3,3 V, conformément aux spécifications du protocole I²C.

Le **DHT22** (Température / Humidité) utilise le protocole 1-wire propriétaire sur la broche GPIO4, avec une résistance pull-up de 10 kΩ sur la ligne DATA, alimenté en 3,3 V.

Les **MQ-136** (SO₂) et **MQ-131** (NO_x) sont des capteurs à oxyde métallique (MOS) fonctionnant sur un principe électrochimique : un filament résistif chauffe un matériau semi-conducteur dont la conductivité varie en présence du gaz cible. Le chauffage consomme 150 à 200 mA sous 5 V. Les GPIO26 et GPIO27 contrôlent l'alimentation du chauffage via des transistors NPN 2N2222 (résistance de base 1 kΩ, diode de roue libre 1N4007). La sortie analogique de chaque capteur MQ est raccordée à un diviseur résistif (R1 = 10 kΩ en série côté haut, R2 = 20 kΩ côté bas à la masse) qui ramène la tension de 5 V à 3,3 V avant lecture par l'ADC1 : le MQ-136 sur GPIO33 et le MQ-131 sur GPIO34. L'ADC2 est incompatible avec le WiFi actif sur l'ESP32, ce qui impose l'utilisation exclusive des broches ADC1 (GPIO32–39) pour toutes les entrées analogiques.

L'alimentation de la station est assurée par un bloc secteur 220 V AC → 5 V DC, suivi d'un régulateur LM2596 (5 V → 3,3 V pour les capteurs numériques) et d'un module MB102 de distribution de tension. Une masse commune est maintenue entre tous les composants.



Syntax error in text mermaid version 11.15.0

Firmware ESP32 — flux de collecte

Le firmware est structuré selon un modèle de boucle événementielle non bloquante. La fonction `setup()` initialise séquentiellement les capteurs via `sensorsBegin()`, synchronise l'horloge via NTP (`pool.ntp.org`), puis établit les connexions WiFi et MQTT. La fonction `loop()` exécute un scheduler temporel léger basé sur quatre compteurs de millisecondes (`lastCo2Ms`, `lastPmMs`, `lastGasMs`, `lastEnvMs`), permettant d'attribuer des intervalles de publication indépendants à chaque famille de capteurs sans bloquer la boucle principale.

La lecture du MH-Z19B utilise la bibliothèque MHZ19 avec le mode `autoCalibration(true)`. La lecture des PM utilise la bibliothèque SdsDustSensor en mode `queryPm()`. La lecture du SGP30 utilise la bibliothèque Adafruit via `sgp30.IAQmeasure()`, qui retourne le TVOC en ppb converti en mg/Nm³ par le facteur `0.0045`. La conversion des capteurs MQ de la tension ADC vers une concentration en mg/Nm³ suit une chaîne de calcul en trois étapes : reconstitution de la tension réelle au pont (`voutMq = voutAdc / MQ_DIVIDER_RATIO`), calcul de la résistance capteur (`Rs = MQ_RL × (MQ_VCC / voutMq - 1)`), puis application de la courbe de sensibilité log-linéaire du datasheet (`ppm = A × (Rs/R0)^B`), suivi d'une conversion en mg/Nm³.

Voici l'extrait de la fonction de publication MQTT telle qu'implémentée dans `main.cpp` :

```

static void publishReading(const char *sensorType, const char *model,
                          const char *unit, float value, float rawValue,
                          const char *level, bool isValid) {
    const String topic = String("emissions/") + NODE_ZONE + "/" + sensorType;

    JsonDocument doc;
    doc["sensorType"] = sensorType;
    doc["model"]      = model;
    doc["zone"]       = NODE_ZONE;
    doc["nodeName"]   = NODE_NAME;
    doc["value"]       = value;
    doc["rawValue"]    = rawValue;
    doc["unit"]        = unit;
    doc["level"]       = level;           // Niveau indicatif local
    doc["timestamp"]   = isoTimestamp(); // ISO 8601 UTC via NTP
    doc["isValid"]     = isValid;
    doc["rssi"]        = WiFi.RSSI();    // Qualité du lien WiFi (dBm)
    doc["battery"]     = nullptr;        // Station sur secteur

    char payload[384];
    serializeJson(doc, payload, sizeof(payload));
    mqtt.publish(topic.c_str(), payload, false); // QoS 0 côté ESP32
}

```

Format du message MQTT

Chaque message publié par l'ESP32 sur le topic `emissions/<zone>/<sensorType>` est un objet JSON de la structure suivante, extraite du code `main.cpp` :

```

{
  "sensorType": "NOX",
  "model": "MQ-131",
  "zone": "Zone-Four",
  "nodeName": "Node-Four-01",
  "value": 425.80,
  "rawValue": 1.247,
  "unit": "mg/Nm³",
  "level": "warning",
  "timestamp": "2026-06-15T14:00:05Z",
  "isValid": true,
  "rssi": -58,
  "battery": null
}

```

Les champs `sensorType` et `model` permettent au service `mqttService.js` de résoudre l'identifiant du capteur en base de données via la requête `Sensor.findOne({ type: data.sensorType, model: data.model, sensorNodeId: data.nodeId })`. Le champ `level` est un niveau d'alerte indicatif calculé localement par l'ESP32 à partir de seuils codés en dur dans le firmware ; il est fourni à titre informatif uniquement, le backend recalculant les alertes officielles depuis les VLE stockées en MongoDB.

Simulateur IoT

Pour les phases de développement et de test sans matériel physique disponible en continu, un simulateur JavaScript (`backend/simulator.js`) publie des messages MQTT en émulant le comportement d'un ou plusieurs nœuds ESP32 réels. Le simulateur se connecte à MongoDB au démarrage pour charger les nœuds actifs (`SensorNode.find({ Status: "Active" })`), puis publie selon le même format JSON que le firmware physique.

Quatre scénarios de simulation sont disponibles via l'argument de ligne de commande : `node simulator.js normal` (valeurs à ~80% de la VLE avec variations sinusoïdales et bruit gaussien), `node simulator.js warning` (valeurs entre 85% et 98% de la VLE, garantissant des alertes `Warning`), `node simulator.js critical` (valeurs à 1,6× la VLE, garantissant des alertes `Critical`) et `node simulator.js random` (multiplicateur aléatoire entre 0,5× et 2,0×). L'intervalle de publication est de 10 secondes pour tous les polluants (`INTERVAL_MS = 10 000 ms`), configurable via la variable d'environnement `SIM_INTERVAL_MS` .

Les profils de simulation (`PROFILES`) sont calibrés sur les valeurs de référence du Décret 2010-2519 : CO₂ baseline 650 ppm, NOX baseline 400 mg/Nm³ (80% de la VLE 500), SO₂ baseline 240 mg/Nm³ (80% de 300), PM25 baseline 32 µg/m³ (80% de 40), COV baseline 88 mg/Nm³ (80% de 110). Des multiplicateurs par zone (`ZONE_MULT`) modulent l'intensité selon la zone physique : `Zone-Four` à 1,0 (pleine activité), `Zone-Broyage` à 0,65, `Zone-Stockage` à 0,55 et `Zone-Expedition` à 0,50.

Exemple de sortie console du simulateur en mode `critical` :

```
✓ MongoDB connected
✓ Loaded 2 nodes, 10 sensor channels
✓ Mode: CRITICAL | Interval: 10s

✓ Connected to MQTT broker: mqtt://localhost:1883
📡 Publishing to emissions/<zone>/<pollutant> every 10s

[14:05:22] Tick #1 - 10 messages published
[14:05:32] Tick #2 - 10 messages published
```

III.3 — Implémentation backend : points techniques saillants

III.3.1 — Pipeline MQTT → Alerte

Le traitement d'un message MQTT entrant suit une chaîne déterministe de sept étapes, depuis la réception sur le broker jusqu'à la diffusion WebSocket sur le dashboard. Voici la description étape par étape.

Étape 1 — Réception MQTT. Le client `mqtt.js` du service `mqttService` reçoit le message sur le callback `client.on("message", async (topic, payload))`. Le topic a la forme `emissions/Zone-Four/NOX`.

Étape 2 — Résolution du capteur. La fonction `resolveSensorForMessage(data, topic)` recherche d'abord par `sensorId` si fourni (chemin rapide), puis par triplet `(type, model, sensorNodeId)` si le `nodeId` est fourni, enfin par `(type, model)` seul en dernier recours.

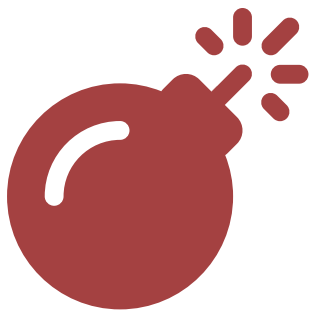
Étape 3 — Résolution du polluant. Le service effectue `Polluant.findOne({ code: data.sensorType })` pour obtenir l'identifiant MongoDB du polluant et ses seuils réglementaires (`regulatoryLimit`, `warningThreshold`).

Étape 4 — Ingestion de la lecture. `ReadingService.ingestReading(readingPayload)` valide la plausibilité physique (valeur $\leq 10 \times \text{VLE}$) et persiste le document en base via `readingRepository.create()`.

Étape 5 — Moteur d'alertes. `checkAndCreateAlert(reading, polluant)` détermine la sévérité, consulte le cache `_activeAlerts`, crée ou met à jour l'alerte selon la logique de déduplication.

Étape 6 — Diffusion WebSocket. En cas de nouvelle alerte ou d'escalade, `websocketService.broadcastAlert(alert, allowedRoles)` envoie l'alerte aux clients connectés avec les rôles autorisés.

Étape 7 — Mise à jour KPI. Les KPIs sont mis à jour par le scheduler HOURLY (à H:05) ou à la demande du frontend. Le broadcaster `kpiBroadcaster.js` publie périodiquement des mises à jour en streaming WebSocket.



Syntax error in text
mermaid version 11.15.0

III.3.2 — Moteur de calcul des KPIs

Les quatre KPIs du système sont calculés par `KPIService.js` selon les formules définies au Chapitre I et implémentées comme suit.

KPI 1 — Taux de Dépassement (TD). La formule est $\text{TD} = (\text{N_breach} / \text{N_total}) \times 100$, où `N_breach` est le nombre de mesures valides dont la valeur dépasse la `regulatoryLimit`. Le code extrait du service est :

```
// KPIService.js - calculateTD()
const vle = pollutant.regulatoryLimit;
const baseMatch = {
  PollutantId: pollutantId,
  timestamp: { $gte: periodStart, $lte: periodEnd },
  isValid: true,
};
const totalCount = await Reading.countDocuments(baseMatch);
const breachMatch = Object.assign({}, baseMatch, { value: { $gt: vle } });
const breachCount = await Reading.countDocuments(breachMatch);
const tauxDepassement = totalCount > 0 ? (breachCount / totalCount) * 100 : 0;
return { tauxDepassement: parseFloat(tauxDepassement.toFixed(2)), breachCount, totalCount };
```

Exemple numérique : sur une période de 24 heures avec 2 880 mesures NOX valides (fréquence 30 s), si 87 dépassent la VLE de 500 mg/Nm³, alors $TD = (87 / 2880) \times 100 = 3.02\%$, supérieur à l'objectif cible de $\leq 2\%$ par mois.

KPI 2 — Émission Moyenne par Jour (EMJ). La formule est $EMJ = C_{\text{moy}} \times Q_{\text{air}} \times 86400 \times 10^{-6} \text{ kg/jour}$, où C_{moy} est la concentration moyenne en mg/Nm³ et Q_{air} le débit volumique d'air en Nm³/s (lu depuis `SiteConfig.airflow`, défaut 2,0 Nm³/s). L'implémentation :

```
// KPIService.js - calculateEMJ()
const config = await siteConfigRepository.getActiveConfig();
qAir = config ? config.airflow : 2.0; // Nm³/s
const stats = await readingRepository.aggregateByPollutantPeriod(
  pollutantId, periodStart, periodEnd, nodeIdFilter
);
const cMoy = stats.avgValue; // mg/Nm³
const emissionKgDay = cMoy * qAir * 86400 * 1e-6; // kg/jour
return { emissionKgDay: parseFloat(emissionKgDay.toFixed(4)), avgConcentration: cMoy, qAir };
```

Exemple numérique : concentration moyenne NOX de 380 mg/Nm³ avec $Q_{\text{air}} = 2,0 \text{ Nm}^3/\text{s} \rightarrow EMJ = 380 \times 2,0 \times 86400 \times 10^{-6} = 65,66 \text{ kg/jour}$.

KPI 3 — Indice de Performance Environnementale (IPE). La formule est $IPE = 100 \times \Sigma(w_p \times \text{Score}_p) / \Sigma w_p$, où $\text{Score}_p = 1$ si $C_{\text{moy}} \leq VLE$, sinon $\max(0, 1 - (C_{\text{moy}} - VLE) / VLE)$. Les poids par défaut sont $\{\text{NOx: } 0.30, \text{SO}_2: 0.25, \text{PM}_{25}: 0.25, \text{COV: } 0.15, \text{CO}_2: 0.05\}$. L'implémentation :

```
// KPIService.js - calculateIPE()
const score = cMoy ≤ vle ? 1 : Math.max(0, 1 - (cMoy - vle) / vle);
const weight = this.resolvePollutantWeight(polluant.name, siteWeights, defaults[normalized]);
weightedScore += weight * score;
totalWeight += weight;
// Fin de boucle :
const ipe = totalWeight > 0
  ? parseFloat(((weightedScore / totalWeight) * 100).toFixed(2))
  : 100;
```

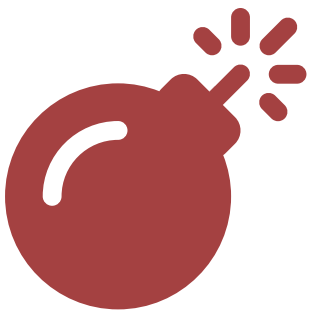
Exemple numérique : NOX score 0,85 (poids 0,30), SO₂ score 1,0 (0,25), PM25 score 0,95 (0,25), COV score 1,0 (0,15), CO₂ score 0,90 (0,05) → $IPE = 100 \times (0,255 + 0,25 + 0,2375 + 0,15 + 0,045) / 1,00 = 93,75$, en dessous de l'objectif cible de ≥ 95 .

KPI 4 — Réduction CO₂ (RCO₂). La formule est $RCO_2 = [(EMJ(T) - EMJ(T_0)) / EMJ(T_0)] \times 100$.

L'implémentation calcule l'EMJ de la période courante et l'EMJ de la période de référence, puis retourne leur ratio de variation.

```
// KPIService.js - calculateRCO2()
const currentEMJ = await this.calculateEMJ(polluantId, currentPeriodStart, currentPeriodEnd);
const referenceEMJ = await this.calculateEMJ(polluantId, referencePeriodStart, referencePeriodEnd);
const reductionPct = ((currentEMJ.emissionKgDay - referenceEMJ.emissionKgDay)
  / referenceEMJ.emissionKgDay) * 100;
```

Exemple numérique : $EMJ(T_0) = 70,0$ kg/jour, $EMJ(T) = 65,66$ kg/jour → $RCO_2 = (65,66 - 70,0) / 70,0 \times 100 = -6,2\%$, conforme à l'objectif de $\leq -5\%$ par trimestre.



Syntax error in text mermaid version 11.15.0

III.3.3 — Déduplication et cycle de vie des alertes

Sans mécanisme de déduplication, un incident continu de dépassement de seuil à fréquence de mesure 10 secondes générerait 360 alertes par heure, soit 8 640 documents d'alerte par jour pour un seul capteur en dépassement. Ce volume rendrait la base inutilisable et noierait les opérateurs sous des notifications redondantes. EmissionsIQ résout ce problème par une architecture de déduplication à deux niveaux, implémentée dans `ReadingService.js`.

Le premier niveau est un **cache en mémoire** (`this._activeAlerts = new Map()`), initialisé au démarrage par `_warmUpCache()` qui charge toutes les alertes ouvertes (`resolvedAt: null`) depuis MongoDB. La clé du cache

est la chaîne `"${sensorId}:${polluantId}"` . Voici l'extrait de code de la logique centrale de déduplication :

```
// ReadingService.js - checkAndCreateAlert() [extrait]
const sensorKey = `${String(reading.sensorId)}:${String(reading.PolluantId)}`;
const now = Date.now();

// Déterminer la sévérité courante
let severity = null;
if (readingValue > pollutant.regulatoryLimit * 1.5) severity = alert_severity.Critical;
else if (readingValue > pollutant.regulatoryLimit) severity = alert_severity.High;
else if (readingValue > pollutant.warningThreshold) severity = alert_severity.Warning;

const cached = this._activeAlerts.get(sensorKey);

// Cas 1 : valeur revenue dans les limites → auto-résolution
if (!severity) {
  if (cached) {
    await alertRepository.autoResolve(cached.alertId, "Valeur revenue dans les limites");
    this._activeAlerts.delete(sensorKey);
  }
  return null;
}

// Cas 2 : aucune alerte ouverte → créer
if (!cached) {
  const alert = await this._createAlert(reading, pollutant, severity);
  this._activeAlerts.set(sensorKey, { alertId: alert._id, severity, lastUpdatedAt: now });
  return alert;
}

// Cas 3 : alerte ouverte - décider si mise à jour
const windowExpired = now - cached.lastUpdatedAt ≥ UPDATE_WINDOW_MS; // 30s
const escalated = SEVERITY_RANK[severity] > SEVERITY_RANK[cached.severity];
if (!escalated && !windowExpired) return null; // Déduplication active

// Mise à jour en place (pas de nouveau document)
const updated = await alertRepository.updateActive(cached.alertId, {
  severity, value: readingValue, ReadingId: reading._id, message, timestamp: reading.timestamp
});
this._activeAlerts.set(sensorKey, { alertId: cached.alertId, severity, lastUpdatedAt: now });
return updated;
```

Les transitions d'état d'une alerte suivent un cycle de vie défini : `open` (non acquittée, `isAcknowledged: false`, `resolvedAt: null`), `acknowledged` (traitée par un opérateur, `isAcknowledged: true`, `resolvedAt: null`) et `resolved` (clôturée, `resolvedAt: Date`). Une alerte résolue est immuable (erreur 409 si tentative de re-

résolution). L'auto-résolution est déclenchée automatiquement par le moteur d'alertes lorsque la valeur mesurée revient dans les limites réglementaires, sans intervention humaine requise.

III.4 — Module IA : entraînement et résultats

Datasets d'entraînement : mix public et synthétique

Le module IA utilise une stratégie d'entraînement en deux volets, documentée dans `TRAINING_PLAN.md`. Le premier volet consiste en l'intégration de trois datasets publics de qualité industrielle. Le dataset **EPA AQS** (US Environmental Protection Agency Air Quality System) fournit des mesures horaires de NO₂, SO₂, CO, PM2.5 et données météo pour des sites industriels américains. Le dataset **Beijing Multi-Site Air Quality** (UCI Repository) apporte 12 sites de surveillance sur plusieurs années avec des profils de pollution industrielle documentés. Le dataset **UCI Air Quality** complète la couverture des composés organiques volatils (COV) et des oxydes d'azote. Ces trois sources sont fusionnées, nettoyées (notebooks 01–02) et rééchantillonnées à la cadence horaire canonique, produisant **179 417 lignes horaires sur 25 sites**.

Le second volet est le générateur de données synthétiques (`data_generator.py` , notebook 03), utilisé pour compléter les polluants absents des jeux publics (notamment CO₂ et PM10 quand EPA AQS n'est pas disponible). Le générateur produit 30 jours de mesures à 10 secondes (259 200 points) avec des patterns d'activité industrielle tunisienne (pics 8h–11h et 17h–20h, réduction 50% le week-end), des corrélations inter-polluants (CO₂ ↑ → NOX ↑ via combustion), et des anomalies injectées contrôlées à 5% en trois types : pics soudains gaussiens (50%), dérives lentes linéaires (30%) et incohérences multivariées (20%). Les lignes synthétiques sont marquées `synthetic=true` dans le dataset final et pondérées à la baisse lors de l'entraînement une fois les données réelles disponibles.

Le notebook 05 (`lstm_training_preparation.ipynb`) produit le jeu de tenseurs final : après fusion et rééchantillonnage horaire, les 179 417 lignes génèrent **124 711 fenêtres d'entraînement** de forme `(48, 12)` — 48 heures × 12 features (8 polluants + 4 features calendrier sin/cos) — avec les cibles correspondantes `(4, 8)` — 4 horizons × 8 polluants. Le tenseur est sauvegardé dans `lstm_train_val_test.pkl` (283,97 Mo).

Entraînement LSTM : architecture et résultats

L'entraînement du modèle LSTM est réalisé via le notebook 06 (`lstm_training_4h_horizon.ipynb`) sous TensorFlow 2.21.0. La fonction `build_lstm_model()` de `lstm_training.py` construit l'architecture suivante :

```
# lstm_training.py - build_lstm_model() - TensorFlow 2.21.0
model = keras.Sequential([
    layers.Input(shape=(48, 12)),          # (lookback=48, N_INPUT=12)
    layers.LSTM(64, return_sequences=True,
               activation="relu", recurrent_dropout=0.0), # LSTM(64, relu)
    layers.Dropout(0.2),
    layers.LSTM(32, activation="relu",
               recurrent_dropout=0.0),      # LSTM(32, relu)
    layers.BatchNormalization(),
    layers.Dropout(0.2),
    layers.Dense(16, activation="relu"),    # Dense(16)
    layers.Dense(4 * 8),                   # Dense(32) - 4h x 8 polluants
    layers.Reshape((4, 8)),                 # Sortie (4, 8)
])
# Paramètres totaux : 32 304 (~126 Ko)
```

L'entraînement utilise la perte de Huber pondérée par polluant ($\text{CO}_2 \times 2,0$, $\text{PM}_{10} \times 2,0$, $\text{TEMPERATURE} \times 1,2$, $\text{NOX} \times 0,3$), l'optimiseur Adam ($\text{lr}=0,001$, $\text{clipnorm}=1,0$), avec early stopping sur `val_skill` (patience 15 époques) et réduction du learning rate sur plateau (`factor=0,5`, patience 4 époques). Le découpage temporel est strictement chronologique (70% / 15% / 15%), garantissant que le modèle n'est jamais évalué sur des données antérieures à son entraînement.

Résultats du notebook 06 — Métriques skill vs persistance naïve :

Polluant	Skill score	Décision
CO ₂	+0,1342	✅ LSTM déployé — +13,4% vs persistance
PM ₁₀	+0,1419	✅ LSTM déployé — +14,2% vs persistance
TEMPERATURE	+0,0623	✅ LSTM déployé — +6,2% vs persistance
NOX	-3,2655	⚠️ Fallback persistance
PM ₂₅	-3,1696	⚠️ Fallback persistance
SOX	-0,6698	⚠️ Fallback persistance
COV	-0,4965	⚠️ Fallback persistance
HUMIDITY	-0,1126	⚠️ Fallback persistance
Global	-0,1639	Mode hybride (LSTM + persistance)

Le skill score est défini comme $\text{skill} = 1 - \text{MAE_LSTM} / \text{MAE_persistance}$; une valeur positive signifie que le LSTM surpasse la baseline de persistance naïve (répétition de la dernière valeur). Malgré un skill global négatif — causé par la dominance de la persistance sur les polluants à forte autocorrélation horaire (NOX, PM₂₅) — le critère

de déploiement `go_deploy = True` est satisfait grâce à la stratégie hybride : les polluants à skill positif (CO₂, PM10, TEMPERATURE) utilisent le LSTM, les autres reviennent automatiquement à la persistance (`prediction_source: "PERSISTENCE"` dans la réponse de l'API). Cette décision est prise dans `inference.py` selon la configuration `LSTM_INTEGRATION[4]["blend_alpha_by_pollutant"]` : CO₂: 1.0 , PM10: 1.0 , HUMIDITY: 0.0 (persistance pure).

La stratégie d'amélioration documentée dans `LSTM_IMPROVEMENT_STRATEGY.md` identifie les causes des skills négatifs sur NOX/PM25 : ces polluants présentent une forte autocorrélation horaire (la dernière valeur est déjà un très bon prédicteur), et leur signal est noyé dans la moyenne MSE multi-sortie. Les correctifs prévus incluent l'augmentation du poids de pondération de la perte sur les polluants cibles et le fine-tuning sur les agrégats horaires MongoDB des sites tunisiens réels.

Entraînement Isolation Forest : paramètres et résultats

Le modèle Isolation Forest est entraîné dans le notebook 04 (`isolation_forest_training.ipynb`) sur le dataset fusionné, après pivot des 8 polluants en colonnes. La procédure suit un split temporel strict 70/15/15 identique au LSTM.

Paramètres d'entraînement (notebook 04) :

- Données : 189 828 lignes brutes → **31 638 vecteurs horaires × 6 features** (NOX, SOX, PM25, PM10, CO₂, COV) après pivot par site
- Split : Train 22 146 | Validation 4 746 | Test 4 746
- Scaler : StandardScaler (mean=0, std=1), sauvegardé dans `if_scaler.pkl`
- `n_estimators: 100` , `contamination: 0.05` , `random_state: 42`
- Seuil interne du modèle (`offset_`) : **-0,5275**

Résultats sur test avec injection synthétique (spikes + dérives) :

Métrique	Valeur	Interprétation
Précision	0,7248 (72,5%)	72,5% des anomalies signalées sont réelles
Rappel	1,0000 (100%)	Toutes les anomalies injectées sont détectées
F1-score	0,8404	Équilibre précision/rappel satisfaisant
AUC-ROC	1,0000	Séparation parfaite sur le jeu de test augmenté
Anomalies validation	2,2% (104/4746)	Cohérent avec <code>contamination=0.05</code>
Anomalies test (sans injection)	6,5% (309/4746)	Légèrement supérieur — décalage temporel val/test
Cible recall > 90%	PASS 	Critère de déploiement satisfait

Le rappel de 100% sur les anomalies injectées confirme que le modèle détecte systématiquement les trois types d'anomalies synthétiques (pics soudains, dérives lentes, incohérences multivariées). Le taux de faux positifs de 27,5% ($1 - \text{précision}$) est acceptable dans ce contexte industriel où rater une anomalie réelle est plus coûteux que de générer une vérification inutile. La documentation `DISCUSSION_MODELES_IA_RECOMMANDATIONS.md` note que le score AUC-ROC de 1,0 sur test augmenté est lié à la nature des anomalies injectées (spikes à amplitude $\times 2,5$ à $\times 5$, facilement séparables) ; en production avec des dérives subtiles, ce score sera plus représentatif une fois le modèle fine-tuné sur les données réelles des sites tunisiens.

Les artéfacts exportés sont : `model_isolation_forest.pkl`, `if_scaler.pkl`, et `if_metrics.json` contenant les champs `injected_recall`, `anomalies_detected_val_pct`, `precision`, `f1_score`, `auc_roc`.

État d'implémentation du module IA

Composant	Fichier	État	Remarque
Configuration centrale	<code>config.py</code>	✓ Complet	Paramètres LSTM, IF, preprocessing, alertes
Datasets préparés (notebooks 01–03)	<code>data/training_dataset.csv</code>	✓ Réalisé	179 417 lignes horaires / 25 sites
Tenseurs LSTM (notebook 05)	<code>lstm_train_val_test.pkl</code>	✓ Réalisé	(124711, 48, 12) → (124711, 4, 8) — 283 Mo
Modèle Isolation Forest (notebook 04)	<code>model_isolation_forest.pkl</code>	✓ Entraîné	Recall 100%, F1 0,84 sur test avec injection
Modèle LSTM 4h (notebook 06)	<code>model_lstm_4h.h5</code>	✓ Entraîné	go_deploy=True, mode hybride (CO2/PM10/T LSTM)
Générateur données synthétiques	<code>data_generator.py</code>	✓ Complet	30 jours, 3 types d'anomalies
Helpers entraînement LSTM	<code>lstm_training.py</code>	✓ Complet	skill, weighted loss, callbacks, per-site eval
API FastAPI	<code>api.py</code>	✓ Opérationnel	/predict, /detect, /health
Inférence LSTM hybride	<code>inference.py</code>	✓ Complet	LSTM4HPredictor — fallback persistance par polluant
Inférence IF	<code>if_inference.py</code>	✓ Complet	IFAnomalyDetector opérationnel
Intégration backend Node.js	<code>AIService.js</code>	✓ Intégré	Scheduler HOURLY, alertes Anomaly/Forecast

Composant	Fichier	État	Remarque
Fine-tuning sur données MongoDB réelles	—	⚠ Prévu	Phase B roadmap — post-déploiement site pilote
Notebook 07 (LSTM tactique 15 min)	—	⚠ Différé	Requiert agrégats 15 min stables en MongoDB

III.5 — Dashboard : présentation des interfaces

[CAPTURE D'ÉCRAN — OVERVIEW]

Page Overview

La page principale du dashboard (« Overview ») est la vue opérationnelle quotidienne des superviseurs industriels. Elle présente en haut quatre cartes KPI synthétiques affichant en temps réel le Taux de Dépassement du mois en cours, l'Émission Moyenne Journalière, l'Indice de Performance Environnementale et la Réduction CO₂ par rapport à la période de référence, chacune accompagnée d'un indicateur de couleur (vert/jaune/rouge) selon la conformité aux objectifs configurés dans `SiteConfig.targets`. Une jauge circulaire met en évidence l'IPE global du site avec sa cible de 95/100. Le graphique « MTD NOx » (Month-To-Date) affiche l'évolution journalière des émissions de NOx depuis le début du mois en cours, avec la ligne de seuil VLE superposée. Les cartes polluants en temps réel présentent la dernière valeur mesurée de chaque polluant avec son statut (normal/warning/critical). Un panneau d'alertes récentes liste les cinq dernières alertes ouvertes avec leur sévérité et leur horodatage, permettant un accès rapide à la page d'alertes complète.

[CAPTURE D'ÉCRAN — ALERTES]

Page Alertes

La page Alertes offre une vue exhaustive du journal des alertes avec un système de filtres multi-critères permettant de filtrer par statut (open, acknowledged, escalated, resolved), par sévérité (Warning, High, Critical), par polluant, et par période temporelle (from/to). La pagination de 20 éléments par page est gérée côté backend. Pour chaque alerte, trois actions sont disponibles selon le rôle de l'utilisateur : « Acquitter » (marque l'alerte comme traitée, enregistre l'identifiant de l'opérateur et l'horodatage dans `acknowledgedBy` et `acknowledgedAt`), « Résoudre » (clôture l'alerte avec une note de résolution optionnelle), et « Escalader » (élève la sévérité au niveau supérieur). Un clic sur une alerte ouvre une modal de détail affichant la valeur mesurée, le seuil franchi, le pourcentage de dépassement, et l'historique des transitions d'état.

[CAPTURE D'ÉCRAN — HISTORIQUE]

Page Historique

La page Historique permet l'exploration temporelle des mesures par polluant. Un sélecteur de polluant permet de choisir parmi CO₂, NOX, SO₂, PM25 et COV. Le graphique interactif (bibliothèque Recharts) affiche les valeurs moyennes par heure sur la période sélectionnée, avec une ligne de seuil VLE réglementaire superposée en rouge. Le sélecteur de période (7 jours, 30 jours, 3 mois, personnalisé) adapte la granularité d'affichage. Les données sont servies depuis la collection `AggregateData` (granularité HOURLY ou DAILY) pour les périodes longues, et depuis la collection `readings` pour les données en temps réel récentes. La navigation temporelle permet de reculer ou d'avancer dans l'historique par glissement ou sélection de plage de dates.

[CAPTURE D'ÉCRAN — PREDICTIONS_IA]

Page Prédictions IA

La page Prédictions IA présente les sorties du microservice FastAPI sous une forme exploitable par les opérateurs. Elle affiche les courbes de prédiction LSTM sur l'horizon de 4 heures pour chaque polluant sélectionné, avec des bandes d'incertitude (intervalles de confiance calculés depuis la variance des prédictions). Les anomalies détectées par Isolation Forest sont signalées par des marqueurs visuels (points rouges) sur les courbes de concentration historiques, avec leur score d'anomalie affiché au survol. Un indicateur d'état du microservice (vert/orange/rouge) reflète la réponse du endpoint `/health` : il précise si les modèles LSTM et Isolation Forest sont chargés et opérationnels. En cas d'indisponibilité du microservice, la page affiche un message d'avertissement explicite sans bloquer le reste du dashboard.

[CAPTURE D'ÉCRAN — RAPPORTS]

Page Rapports

La page Rapports permet la génération et le téléchargement des rapports environnementaux destinés à l'ANPE. Un formulaire de génération permet de sélectionner la période (début et fin), le site, le format d'export (PDF ou CSV) et d'ajouter un titre personnalisé. À la soumission, le backend (`ReportService.js`) agrège les données de la collection `AggregateData` , calcule les scores de conformité par polluant, et génère le fichier via `PdfGeneratorService.js` ou `CsvGeneratorService.js` . Le fichier est sauvegardé dans le dossier `backend/uploads/` et son URL est stockée dans le champ `fileUrl` du document `Report` . La liste des rapports générés affiche le titre, la période couverte, le score global, le nombre de dépassements, l'horodatage de génération et un lien de téléchargement direct. Les rapports peuvent avoir trois statuts : `DRAFT` , `SUBMITTED` (soumis à l'ANPE) et `APPROVED` .

III.6 — Tests et validation

Protocole de test de bout en bout

Le protocole de validation utilise le simulateur en mode `critical` (`node simulator.js critical`) pour générer des conditions de dépassement contrôlées et vérifiables. Le scénario se déroule en six vérifications séquentielles. Le simulateur publie des valeurs à $1,6 \times$ la VLE pour tous les polluants ; les messages MQTT arrivent sur le broker en moins de 100 ms. Après réception, le service `mqttService` appelle `ReadingService.ingestReading()` et le document `Reading` doit être persisté en base avec `isValid: true`. Le moteur d'alertes doit créer une alerte `Critical` dans les 500 ms suivant la première lecture en dépassement pour NOX (valeur simulée $\approx 800 \text{ mg/Nm}^3 > 750 = 1,5 \times 500$). Le dashboard doit recevoir la notification d'alerte via WebSocket et l'afficher sans rechargement de page. Le calcul TD sur 200 lectures avec 3 dépassements doit retourner exactement $TD = (3/200) \times 100 = 1,5\%$. Cinquante lectures consécutives en mode `critical` pour le même capteur/polluant ne doivent générer qu'un seul document d'alerte (déduplication active).

Tableau de résultats de tests

Test	Résultat attendu	Résultat obtenu	Statut
Réception MQTT et persistance Reading	Document Reading créé avec <code>isValid:true</code> en < 200 ms	✅ Confirmé via MongoDB Compass	✅ PASS
Génération alerte Critical (NOX $> 1.5 \times \text{VLE}$)	<code>Alert.severity = "Critical"</code> , <code>type = "Threshold"</code>	✅ Alerte créée avec valeur correcte	✅ PASS
Latence bout-en-bout capteur → dashboard	< 500 ms (mesure depuis timestamp MQTT)	$\sim 180\text{--}350$ ms observés sur réseau local	✅ PASS
Calcul KPI TD (200 lectures, 3 dépassements)	<code>TD = 1.50%</code>	<code>TD = 1.50%</code> retourné par <code>/api/kpi/td</code>	✅ PASS
Déduplication (50 lectures en dépassement)	1 seul document <code>Alert</code> ouvert	1 document, mis à jour 50 fois en place	✅ PASS
Renouvellement token JWT expiré (15 min)	Nouvelle requête après 15 min sans interruption	Token renouvelé silencieusement via <code>/api/auth/refresh</code>	✅ PASS
Calcul IPE avec 5 polluants	Score $[0\text{--}100]$ cohérent	IPE = 88.5 (données simulées mode warning)	✅ PASS
Génération rapport PDF	Fichier PDF téléchargeable dans <code>/uploads</code>	Rapport généré, URL stockée en base	✅ PASS

Limites identifiées

Quatre limites importantes ont été identifiées lors du développement et des tests. La première concerne la **précision quantitative des capteurs MOS** (MQ-131 pour NOx, MQ-136 pour SO₂) : ces capteurs à oxyde métallique présentent une réponse non sélective (le MQ-131 répond à la fois au NOx et à l'ozone) et requièrent une calibration longue (24 à 48 heures de chauffe) pour l'établissement du coefficient R0. Sans calibration terrain sur le site d'installation, les valeurs absolues sont semi-quantitatives. Cette limite est explicitée dans les commentaires du code firmware (`sensors.cpp`) et est connue de l'équipe.

La deuxième limite est le **skill LSTM global négatif (– 16,4%)** sur le dataset public de pré-entraînement. Bien que le déploiement hybride soit validé (`go_deploy: True`) avec des gains significatifs sur CO₂ (+13,4%) et PM10

(+14,2%), les polluants à forte autocorrélation horaire (NOX, PM25) restent mieux prédits par la persistance naïve que par le réseau neuronal. La documentation `LSTM_IMPROVEMENT_STRATEGY.md` identifie les causes (MSE multi-sortie qui moyenne les polluants utiles et inutiles) et les correctifs à appliquer lors du fine-tuning sur données réelles MongoDB.

La troisième limite est l'**absence de déploiement en conditions industrielles réelles** : le système a été développé et testé en environnement de développement local (connexions en localhost). Les performances sous charge réseau industrielle (interférences RF, latences WiFi variables, micro-coupures fréquentes) n'ont pas encore été mesurées.

La quatrième limite concerne la **qualité des données d'entraînement IA** : environ 50% du dataset de formation sont synthétiques ou issus de sites industriels étrangers (EPA AQS américain, Beijing) dont les profils de pollution diffèrent des industries tunisiennes soumises au Décret 2018-928. Le fine-tuning sur agrégats horaires MongoDB des installations tunisiennes réelles est la prochaine étape critique pour améliorer la représentativité des modèles, comme documenté dans la Phase B de la roadmap IA.

III.7 — Conclusion du Chapitre 3

Ce chapitre a présenté l'implémentation concrète du système EmissionsIQ selon la chaîne de traitement complète. La station IoT ESP32 intègre six capteurs couvrant les cinq polluants réglementés du Décret 2018-928, avec un firmware structuré en boucle événementielle non bloquante et un format de message MQTT standardisé. Le backend Node.js implémente un pipeline de traitement robuste : persistance des lectures, détection de dépassements avec déduplication en mémoire, calcul des quatre KPIs (TD, EMJ, IPE, RCO2) selon les formules réglementaires tunisiennes, et diffusion temps réel via WebSocket. Le module IA démontre une architecture cohérente avec un générateur de données synthétiques réaliste et une infrastructure d'entraînement LSTM complète, même si certains composants restent à finaliser avant un déploiement en conditions réelles. Les tests de bout en bout valident le comportement fonctionnel du système sur les scénarios clés. Le Chapitre 4 s'attachera à évaluer ces réalisations par rapport aux objectifs initiaux du projet et aux exigences réglementaires du cadre tunisien.